1. Implementation of symbol table.

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit symboltable.c

```
#include<stdio.h>
#include<math.h>
#include<string.h>
#include<ctype.h>
#include<stdlib.h>
void main()
{
 int x=0, n, i=0,j=0,flag=0;
 void *mypointer,*address[5];
 char ch,Search,Array2[15],Array3[15],c;
 printf("Input the expression ending with $ sign:");
 while((c=getchar())!='$')
 {
  Array2[i]=c;
  i++;
 }
 n=i-1;
 printf("Given Expression:");
 i=0;
 while(i<=n)
 {
  printf("%c",Array2[i]);
  i++;
 }
 printf("\n Symbol Table display\n");
 printf("Symbol \t addr \t type");
 while(j<=n)
 {
  c=Array2[j];
  if(isalpha(c))
  {
   mypointer=malloc(c);
   address[x]=mypointer;
   Array3[x]=c;
   printf("\n%c \t %p \t identifier\n",c,mypointer);
   x++;
   j++;
  }
  else if(c=='+'||c=='-'||c=='*'||c=='=')
   {
    mypointer=malloc(c);
    address[x]=mypointer;
    Array3[x]=c;
    printf("\n %c \t %p \t operator\n",c,mypointer);
    x++;
    j++;
   }
   else
```

```
   {
        if(isdigit(c))
   {
    mypointer=malloc(c);
    address[x]=mypointer;
    Array3[x]=c;
    printf("\n %c \t %p \t Constant\n",c,mypointer);
    x++;
    j++;
   }
   }
   }
printf("\nEnter the symbol to be searched\n");
scanf(" %c",&ch);
printf("\nThe Symbol forsearching is:%c\n",ch);
for(i=0;i<=x;i++)
{
if(ch==Array3[i])
{
printf("symbol found\n");
printf("%c\t%s\t%p\n",ch,"@address",address[i]);
flag=1;
}
}
if(flag==0)
printf("symbol not found\n");
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gcc symboltable.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Input the expression ending with $ sign:a+9$
Given Expression:a+9
 Symbol Table display
Symbol          addr    type
a       0x5caa0a7cbac0      identifier

+       0x5caa0a7cbb30      operator

9       0x5caa0a7cbb70      Constant

Enter the symbol to be searched
a

The Symbol forsearching is:a
symbol found
a       @address        0x5caa0a7cbac0
```

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Input the expression ending with $ sign:a+b$
Given Expression:a+b
 Symbol Table display
Symbol          addr    type
a       0x62b4eefb9ac0      identifier

 +      0x62b4eefb9b30      operator

b       0x62b4eefb9b70      identifier

Enter the symbol to be searched
+

The Symbol forsearching is:+
symbol found
+       @address        0x62b4eefb9b30
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

2. Develop a lexical analyzer to recognize a few patterns inc (ex. Identifiers, constants, comments, operators etc.)

Creation of Source Program: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit tokens.l

```
%%
[a-zA-Z][a-zA-Z0-9]* {printf("\n %s is Identifier", yytext);}
[0-9]+ {printf("\n %s is Constant", yytext);}
">"|"<"|">="|"<="|"=="|"!=" {printf("\n %s is Relational operator", yytext);}
"+"|"-"|"*"|"/"|"%" {printf("\n %s is Arithmetic operator", yytext);}
"/*".*"*/" {printf("\n %s is Comment", yytext);}
%%
int main()
{
yylex();
return(0);
}
int yywrap()
{
return 1;
}
```

Execution:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex  token.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
/*jkdskjdjk*/

 /*jkdskjdjk*/ is Comment
23

 23 is Constant
a+b

 a is Identifier
 + is Arithmetic operator
 b is Identifier
^Z
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

3. Implementation of lexical analyzer using lex tool.

Creation of Source Program: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit lexicalanalyzer.l

```
%{
int COMMENT=0;
%}
identifier [a-zA-Z][a-zA-Z0-9]*
%%
#.* { printf("\n%s is a PREPROCESSOR DIRECTIVE",yytext);}
int |
float |
main |
char |
double |
while |
for |
do |
if |
break |
continue |
void |
switch |
case |
long |
struct |
const |
typedef |
return |
else |
getch {printf("\n\t%s is a KEYWORD",yytext);}
"/*" {COMMENT = 1;}
"*/" {COMMENT = 0;}
{identifier}\( {if(!COMMENT)printf("\n\nFUNCTION\n\t%s",yytext);}
\{ {if(!COMMENT) printf("\n BLOCK BEGINS");}
\} {if(!COMMENT) printf("\n BLOCK ENDS");}
{identifier}(\[[0-9]*\])? {if(!COMMENT) printf("\n %s IDENTIFIER",yytext);}
\".*\" {if(!COMMENT) printf("\n\t%s is a STRING",yytext);}
[0-9]+ {if(!COMMENT) printf("\n\t%s is a NUMBER",yytext);}
\)(\;)? {if(!COMMENT) printf("\n\t");ECHO;printf("\n");}
\( ECHO;
= {if(!COMMENT)printf("\n\t%s is an ASSIGNMENT OPERATOR",yytext);}
\<= |
\>= |
\< |
== |
\> {if(!COMMENT) printf("\n\t%s is a RELATIONAL OPERATOR",yytext);}
\+ |
\- |
\* |
\\ {if(!COMMENT) printf("\n\t%s is a arithmetic operator",yytext); }
```

```
%%
int main(int argc,char **argv)
{
if(argc>1)
{
FILE *file;
file=fopen(argv[1],"r");
if(!file)
{
printf("\n Could not open the file:%s",argv[1]);
exit(0);
}
yyin=file;
}
yylex();
printf("\n\n");
return 0;
}
int yywrap()
{
return 0;
}
```

Create a file var.c before execution:

```
/* c program to print a message */
#include<stdio.h>
int main()
{
int a,b;
if(a>b)
printf("hello world");
return 0;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex lexicalanalyzer.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out var.c
```

#include<stdio.h> is a PREPROCESSOR DIRECTIVE

int is a KEYWORD

FUNCTION
main(
)

BLOCK BEGINS

int is a KEYWORD
a IDENTIFIER,

```
 b IDENTIFIER;


FUNCTION
        if(
 a IDENTIFIER
        > is a RELATIONAL OPERATOR
 b IDENTIFIER
        )



FUNCTION
        printf(
        "hello world" is a STRING
        );


        return is a KEYWORD
        0 is a NUMBER;

 BLOCK ENDS

^Z
[6]+  Stopped                 ./a.out var.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

4. Generate yacc specification for a few syntactic categories.
a) Program to recognize a valid arithmetic expression that uses operator +,-, * and /.

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit arithmetic_id.l

```
%{
/* This LEX program returns the tokens for the expression */
#include "y.tab.h"
%}
%%
"=" {printf("\n Operator is EQUAL");}
"+" {printf("\n Operator is PLUS");}
"-" {printf("\n Operator is MINUS");}
"/" {printf("\n Operator is DIVISION");}
"*" {printf("\n Operator is MULTIPLICATION");}
[a-zA-Z][a-zA-Z0-9]* {printf("\n Identifier is %s",yytext);return ID;}
. return yytext[0];
\n return 0;
%%
int yywrap()
{
return 1;
}
```

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit arithmetic_id.y

```
%{
#include<stdio.h>
/* This YYAC program is for recognizing the Expression */
int yylex(void);

void yyerror(char *s);
%}
%token ID
%%
statement: E{printf("\n Valid arithmetic expression"); $$ = $1;}
             ;
E        : E '+' ID
         | E '-' ID
         | E '*' ID
         | E '/' ID
         | ID
         ;
%%
```

```
extern FILE *yyin;
int main()
{
do
{
yyparse();
}while(!feof(yyin));
return 0;
}
void yyerror(char *s)
{

}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lec arithmetic_id.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ yacc -dy arithmetic_id.y
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc  lex.yy.c  y.tab.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
a+b;
Identifier is a
Operator is PLUS
Identifier is b
```

b) Program to recognize a valid variable which starts with a letter followed by any number of letter or digits.

Creation:nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit var_id.l

```
%{
/* This LEX program returns the tokens for the Expression */
#include "y.tab.h"
%}
%%
"int" {return INT;}
"float" {return FLOAT;}
"double" {return DOUBLE;}
[a-zA-Z][a-zA-Z0-9]* {printf("\nIdentifier is %s",yytext);return ID;}
. return yytext[0];
\n return 0;
%%
int yywrap()
{
return 1;
}
```

Creation:nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit var_id.y

```
%{
#include<stdio.h>
/* This YYAC program is for recognizing the Expression */
int yylex(void);

void yyerror(char *s);
%}
%token ID INT FLOAT DOUBLE
%%
D       :T L
        ;
L       :L ','ID
        |ID
        ;
T       :INT
        |FLOAT
        |DOUBLE
        ;
%%
```

```
extern FILE *yyin;
int main()
{
do
{
yyparse();
}while(!feof(yyin));
return 0;
}
void yyerror(char *s)
{

}
```

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex var_id.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ yacc -dy var_id.y
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c y.tab.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
int a1,b12

Identifier is a1
Identifier is b12^Z
[1]+  Stopped                 ./a.out
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

c) Implementation of calculator using lex and yacc.

Creation : nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit cal.l

```
%{
#include<stdio.h>
#include "y.tab.h"
extern int yylval;
%}
%%
[0-9]+ { yylval=atoi(yytext); return NUMBER;}
[\n] return 0;
[\t] ;
. return yytext[0];
%%
int yywrap()
{
return 0;
}
```

Creation : nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit cal.y

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex(void);
int yyerror(char *s);
%}
%token NAME NUMBER
%left GE LE NE EQ '<">"%'
%left '-"+'
%left '*"/'
%nonassoc UMINUS
%%
Statement :NAME '=' exp
        |exp {printf("=%d\n",$1);}
;
exp     : NUMBER {$$ == $1;}
        | exp '+' exp {$$ =$1 +$3;}
        | exp '-' exp {$$ =$1 - $3;}
        | exp '*' exp {$$ =$1 * $3;}
        | exp '/' exp {$$ =$1 / $3;}
        | exp '-' exp %prec UMINUS {$$ =-$2;}
        | '('exp')' {$$ =$2;}
        ;
%%


int main()
```

```
{
printf("Enter the expression:");
yyparse();
}
int yyerror(char *s)
{
printf("\n Error Occurred\n");
return 1;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex cal.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ yacc -dy cal.y
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c y.tab.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the expression:2+3
=5
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the expression:2+3*7
=23
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

5. Convert the bnf rules into yacc form and write code to generate abstract syntax tree.

nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit bnf.l

```
%{
#include"y.tab.h"
#include<stdio.h>
#include<string.h>
int LineNo=1;
%}
identifier [a-zA-Z][_a-zA-Z0-9]*
number [0-9]+|([0-9]*\.[0-9]+)
%%
main\(\) return MAIN;
if return IF;
else return ELSE;
while return WHILE;
int |
char |
float return TYPE;
{identifier} {strcpy(yylval.var,yytext);
return VAR;}
{number} {strcpy(yylval.var,yytext);
return NUM;}
\< |
\> |
\>= |
\<= |
```

```
== {strcpy(yylval.var,yytext);

return RELOP;}

[ \t] ;

\n LineNo++;

. return yytext[0];

%%



nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit bnf.y

%{

#include<string.h>
#include<stdio.h>
#include<stdlib.h>

struct quad

{

char op[5];

char arg1[10];

char arg2[10];

char result[10];

}QUAD[30];

struct stack

{

int items[100];

int top;

}stk;

int Index=0,tIndex=0,StNo,Ind,tInd;

extern int LineNo;

%}
```

```
%union
{
char var[10];
}
%token <var> NUM VAR RELOP
%token MAIN IF ELSE WHILE TYPE
%type <var> EXPR ASSIGNMENT CONDITION IFST ELSEST WHILELOOP
%left '-' '+'
%left '*' '/'
%%
PROGRAM : MAIN BLOCK
;
BLOCK: '{' CODE '}'
;
CODE: BLOCK
| STATEMENT CODE
| STATEMENT
;
STATEMENT: DESCT ';'
| ASSIGNMENT ';'
| CONDST
| WHILEST
;
DESCT: TYPE VARLIST
;
VARLIST: VAR ',' VARLIST
```

```
| VAR
;
ASSIGNMENT: VAR '=' EXPR{
strcpy(QUAD[Index].op,"=");
strcpy(QUAD[Index].arg1,$3);
strcpy(QUAD[Index].arg2,"");
strcpy(QUAD[Index].result,$1);
strcpy($$,QUAD[Index++].result);
}
;
EXPR: EXPR '+' EXPR {AddQuadruple("+",$1,$3,$$);}
| EXPR '-' EXPR {AddQuadruple("-",$1,$3,$$);}
| EXPR '*' EXPR {AddQuadruple("*",$1,$3,$$);}
| EXPR '/' EXPR {AddQuadruple("/",$1,$3,$$);}
| '-' EXPR {AddQuadruple("UMIN",$2,"",$$);}
| '(' EXPR ')' {strcpy($$,$2);}
| VAR
| NUM


;
CONDST: IFST{
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
```

```
}
| IFST ELSEST
;
IFST: IF '(' CONDITION ')' {
strcpy(QUAD[Index].op,"==");
strcpy(QUAD[Index].arg1,$3);
strcpy(QUAD[Index].arg2,"FALSE");
strcpy(QUAD[Index].result,"-1");
push(Index);
Index++;
}
BLOCK { strcpy(QUAD[Index].op,"GOTO"); strcpy(QUAD[Index].arg1,"");
strcpy(QUAD[Index].arg2,"");
strcpy(QUAD[Index].result,"-1");
push(Index);
Index++;
};
ELSEST: ELSE{
tInd=pop();
Ind=pop();
push(tInd);
sprintf(QUAD[Ind].result,"%d",Index);
}
BLOCK{
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
```

```
};
CONDITION: VAR RELOP VAR {AddQuadruple($2,$1,$3,$$);
StNo=Index-1;
}
| VAR
| NUM
;
WHILEST: WHILELOOP{
Ind=pop();
sprintf(QUAD[Ind].result,"%d",StNo);
Ind=pop();
sprintf(QUAD[Ind].result,"%d",Index);
}
;
WHILELOOP: WHILE('CONDITION ')' {
strcpy(QUAD[Index].op,"==");
strcpy(QUAD[Index].arg1,$3);
strcpy(QUAD[Index].arg2,"FALSE");



strcpy(QUAD[Index].result,"-1");
push(Index);
Index++;
}
BLOCK {
strcpy(QUAD[Index].op,"GOTO");
```

```
strcpy(QUAD[Index].arg1,"");

strcpy(QUAD[Index].arg2,"");

strcpy(QUAD[Index].result,"-1");

push(Index);

Index++;

}

;

%%

extern FILE *yyin;

int main(int argc,char *argv[])

{

FILE *fp;

int i;

if(argc>1)

{

fp=fopen(argv[1],"r");

if(!fp)

{

printf("\n File not found");

exit(0);

}

yyin=fp;

}

yyparse();

printf("\n\n\t\t ----------------------------""\n\t\t Pos Operator \tArg1 \tArg2 \tResult" "\n\t\t--------------
------");
```

```
for(i=0;i<Index;i++)
{
printf("\n\t\t %d\t %s\t %s\t %s\t%s",i,QUAD[i].op,QUAD[i].arg1,QUAD[i].arg2,QUAD[i].result);
}
printf("\n\t\t ----------------------");
printf("\n\n"); return 0; }
void push(int data)
{ stk.top++;
if(stk.top==100)
{
printf("\n Stack overflow\n");
exit(0);
}
stk.items[stk.top]=data;
}
int pop()
{
int data;


if(stk.top==-1)
{
printf("\n Stack underflow\n");
exit(0);
}
data=stk.items[stk.top--];
```

```
return data;

}

void AddQuadruple(char op[5],char arg1[10],char arg2[10],char result[10])

{

strcpy(QUAD[Index].op,op);

strcpy(QUAD[Index].arg1,arg1);

strcpy(QUAD[Index].arg2,arg2);

sprintf(QUAD[Index].result,"t%d",tIndex++);

strcpy(result,QUAD[Index++].result);

}

yyerror()

{

printf("\n Error on line no:%d",LineNo);

}
```

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit test.c

```
main()
{
int a,b,c;
if(a<b)
{
a=a+b;
}
while(a<b)
{
a=a+b;
}
if(a<=b)
{
c=a-b;
}
else
{
c=a+b;
}
}
```

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex bnf.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ yacc -d bnf.y
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c y.tab.c -ll -lm -w
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out test.c
```

Pos	Operator	Arg1	Arg2	Result
0	<	a	b	t0
1	==	t0	FALSE	5
2	+	a	b	t1
3	=	t1		a
4	GOTO			5
5	<	a	b	t2
6	==	t2	FALSE	10
7	+	a	b	t3
8	=	t3		a
9	GOTO			5
10	<=	a	b	t4
11	==	t4	FALSE	15
12	-	a	b	t5
13	=	t5		c
14	GOTO			17
15	+	a	b	t6
16	=	t6		c

---

6. Implement type checking

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit typecheck.c

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
int n,i,k,flag=0;
char vari[15],typ[15],b[15],c;
printf("Enter the number of variables:");
scanf(" %d",&n);
for(i=0;i<n;i++)
{
printf("Enter the variable[%d]:",i);
scanf(" %c",&vari[i]);
printf("Enter the variable-type[%d](float-f,int-i):",i);
scanf(" %c",&typ[i]);
if(typ[i]=='f')
flag=1;
}
printf("Enter the Expression(end with $):");
i=0;
getchar();
while((c=getchar())!='$')
{
b[i]=c;
i++;  }
k=i;
for(i=0;i<k;i++)
{
if(b[i]=='/')
{
flag=1;
break;  }  }
for(i=0;i<n;i++)
{
if(b[0]==vari[i])
{
if(flag==1)
{
if(typ[i]=='f')
{  printf("\nthe datatype is correctly defined..!\n");
break;  }
else
{  printf("Identifier %c must be a float type..!\n",vari[i]);
break;  }  }
else
{  printf("\nthe datatype is correctly defined..!\n");
break;  }  }
}
```

```
return 0;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc typecheck.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the number of variables:2
Enter the variable[0]:a
Enter the variable-type[0](float-f,int-i):i
Enter the variable[1]:b
Enter the variable-type[1](float-f,int-i):f
Enter the Expression(end with $):a+b$
Identifier a must be a float type..!
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the number of variables:2
Enter the variable[0]:a
Enter the variable-type[0](float-f,int-i):i
Enter the variable[1]:b
Enter the variable-type[1](float-f,int-i):i
Enter the Expression(end with $):a*b$

the datatype is correctly defined..!
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

7. Implement any one storage allocation strategies (heap, stack, static)

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit heap.c

```
#include<stdio.h>
#include<stdlib.h>
#define TRUE 1
#define FALSE 0
typedef struct Heap
{
int data;
struct Heap *next;
}
node;
node *create();
void main()
{
int choice,val;
char ans;
node *head;
void display(node *);
node *search(node *,int);
node *insert(node *);
void dele(node **);
head=NULL;
do
{
printf("\nprogram to perform various operations on heap using dynamic memory management");
printf("\n1.create");
printf("\n2.display");
printf("\n3.insert an element in a list");
printf("\n4.delete an element from list");
printf("\n5.quit");
printf("\nenter your chioce(1-5)");
scanf("%d",&choice);
switch(choice)
{
case 1:head=create();
break;
case 2:display(head);
break;
case 3:head=insert(head);
break;
case 4:dele(&head);
break;
case 5:exit(0);
default:
printf("invalid choice,try again");
}
}while(choice!=5);
}
```

```
node* create()
{
node *temp,*New,*head;
int val,flag;
char ans='y';
node *get_node();
temp=NULL;
flag=TRUE;
do
{
printf("\n enter the element:");
scanf(" %d",&val);
New=get_node();
if(New==NULL)
printf("\nmemory is not allocated");
New->data=val;
if(flag==TRUE)
{
head=New;
temp=head;
flag=FALSE;
}
else
{
temp->next=New;
temp=New;
}
printf("\ndo you want to enter more elements?(y/n)");
scanf(" %c", &ans);
}while(ans=='y');
printf("\nthe list is created\n");
return head;
}
node *get_node()
{
node *temp;
temp=(node*)malloc(sizeof(node));
temp->next=NULL;
return temp;
}
void display(node *head)
{
node *temp;
temp=head;
if(temp==NULL)
{
printf("\nthe list is empty\n");
return;
}
while(temp!=NULL)
{
printf("%d->",temp->data);
```

```
temp=temp->next;
}
printf("NULL");
}
node *search(node *head,int key)
{
node *temp;
int found;
temp=head;
if(temp==NULL)
{
printf("the linked list is empty\n");
return NULL;
}
found=FALSE;
while(temp!=NULL && found==FALSE)
{
if(temp->data!=key)
temp=temp->next;
else
found=TRUE;
}
if(found==TRUE)
{
printf("\nthe element is present in the list\n");
return temp;
}
else
{
printf("the element is not present in the list\n");
return NULL;
}
}
node *insert(node *head)
{
int choice;
node *insert_head(node *);
void insert_after(node *);
void insert_last(node *);
printf("\n1.insert a node as a head node");
printf("\n2.insert a node as a last node");
printf("\n3.insert a node at intermediate position in the list");
printf("\nenter your choice for insertion of node:");
scanf("%d",&choice);
switch(choice)
{
case 1:head=insert_head(head);
break;
case 2:insert_last(head);
break;
case 3:insert_after(head);
break;
```

```
}
return head;
}
node *insert_head(node *head)
{
node *New,*temp;
New=get_node();
printf("\nEnter the element which you want to insert");
scanf("%d",&New->data);
if(head==NULL)
head=New;
else
{
temp=head;
New->next=temp;
head=New;
}
return head;
}
void insert_last(node *head)
{
node *New,*temp;
New=get_node();
printf("\nenter the element which you want to insert");
scanf("%d",&New->data);
if(head==NULL)
head=New;
else
{
temp=head;
while(temp->next!=NULL)
temp=temp->next;
temp->next=New;
New->next=NULL;
}
}
void insert_after(node *head)
{
int key;
node *New,*temp;
New=get_node();
printf("\nenter the elements which you want to insert");
scanf("%d",&New->data);
if(head==NULL)
{
head=New;
}
else
{
printf("\enter the element which you want to insert the node");
scanf("%d",&key);
temp=head;
```

```
do
{
if(temp->data==key)
{
New->next-temp->next;
temp->next=New;
return;
}
else
temp=temp->next;
}
while(temp!=NULL);
}
}
node *get_prev(node *head,int val)
{
node *temp,*prev;
int flag;
temp=head;
if(temp==NULL)
return NULL;
flag=FALSE;
prev=NULL;
while(temp!=NULL && ! flag)
{
if(temp->data!=val)
{
prev=temp;
temp=temp->next;
}
else
flag=TRUE;
}
if(flag)
return prev;
else
return NULL;
}
void dele(node **head)
{
node *temp,*prev;
int key;
temp=*head;
if(temp==NULL)
{
printf("\nthe list is empty\n");
return;
}
printf("\nenter the element you want to delete:");
scanf("%d",&key);
temp=search(*head,key);
if(temp!=NULL)
```

```
{
prev=get_prev(*head,key);
if(prev!=NULL)
{
prev->next=temp->next;
free(temp);
}
else
{
*head=temp->next;
free(temp);
}
printf("\nthe element is deleted\n");
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc heap.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out

program to perform various operations on heap using dynamic memory management
1.create
2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)1

 enter the element:12

do you want to enter more elements?(y/n)y

 enter the element:45

do you want to enter more elements?(y/n)y

 enter the element:45

do you want to enter more elements?(y/n)n

the list is created

program to perform various operations on heap using dynamic memory management
1.create
2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)2
12->45->45->NULL
program to perform various operations on heap using dynamic memory management
1.create
```

2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)3
1.insert a node as a head node
2.insert a node as a last node
3.insert a node at intermediate position in the list
enter your choice for insertion of node:1

Enter the element which you want to insert56

program to perform various operations on heap using dynamic memory management
1.create
2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)2
56->12->45->45->NULL
program to perform various operations on heap using dynamic memory management
1.create
2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)4

enter the element you want to delete:45

the element is present in the list

the element is deleted

program to perform various operations on heap using dynamic memory management
1.create
2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)2
56->12->45->NULL
program to perform various operations on heap using dynamic memory management
1.create
2.display
3.insert an element in a list
4.delete an element from list
5.quit
enter your chioce(1-5)5

8. Write a lex program to count the number of words and number of lines in a given file or program.

Creation of Source Program: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit count.l

```
%{
 #include <stdio.h>
int word_count = 0;
int line_count = 0;
int char_count = 0;
%}
%%
"\n" { line_count++; word_count++;char_count++; }
[ \t]+ { char_count++; word_count++;}
. { char_count++; }
%%
int main()
{
printf("Enter text (Press Ctrl+D to end input):\n");
yylex();
printf("\n--- Results ---\n");
printf("Lines: %d\n", line_count);
printf("Words: %d\n", word_count);
printf("Characters: %d\n", char_count);
return 0;
}
int yywrap()
{
return 1;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex count.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter text (Press Ctrl+D to end input):
hello
welcome to NNRG

--- Results ---
Lines: 2
Words: 4
Characters: 22
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

9. Write a ‘C’ program to implement lexical analyzer using c program.

Creation of Source Program: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit lexicalanalyzer.c

```
#include<string.h>
 #include<ctype.h>
 #include<stdio.h>
 void keyword(char str[10])
{
if(strcmp("for",str)==0||strcmp("while",str)==0||strcmp("do",str)==0||strcmp("int",
str )==0||strcmp("float",str)==0||strcmp("char",str)==0||strcmp("double",str)==0||strcmp("static",str)
==0||strcmp("switch",str)==0||strcmp("case",str)==0)
printf("\n%s is a keyword",str);
else
printf("\n%s is an identifier",str);
 }
void main()
{
FILE *f1,*f2,*f3;
char c, str[10], st1[10];   int num[100], lineno=0, tokenvalue=0,i=0,j=0,k=0;
 printf("\n Enter the c program : ");/*gets(st1);*/
 f1=fopen("input","w");
 while((c=getchar())!=EOF)
    putc(c,f1);
    fclose(f1);
f1=fopen("input","r");
f2=fopen("identifier","w");
f3=fopen("specialchar","w");
while((c=getc(f1))!=EOF)
{
  if(isdigit(c))
{
tokenvalue=c-'0';
c=getc(f1);
while(isdigit(c))
{
  tokenvalue*=10+c-'0';
  c=getc(f1);
   }
num[i++]=tokenvalue;
 ungetc(c,f1);
}
else if(isalpha(c))
{
putc(c,f2);
 c=getc(f1);
 while(isdigit(c)||isalpha(c)||c=='_'||c=='$')
    {
    putc(c,f2);
    c=getc(f1);
       }
        putc(' ',f2);
```

```
        ungetc(c,f1);
}
else  if(c==' '||c=='\t')
   printf(" ");
   else if(c=='\n')
   lineno++;
   else
   putc(c,f3);
}
fclose(f2);
fclose(f3);
fclose(f1);
printf("\n The no's in the program are :");
for(j=0; j<i; j++)
printf("%d", num[j]);
printf("\n");
f2=fopen("identifier", "r");
k=0;
printf("The keywords and identifiers are:");
while((c=getc(f2))!=EOF)
{
    if(c!=' ')
    str[k++]=c;
    else
      {
       str[k]='\0';
       keyword(str);
       k=0;
       }
}
fclose(f2);
f3=fopen("specialchar","r");
printf("\n Special characters are : ");
while((c=getc(f3))!=EOF)
printf("%c",c);
printf("\n");
fclose(f3);
printf("Total no. of lines are:%d", lineno);
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lexicalanalyzer.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out

 Enter the c program : main()
{
int a,b;
a=b+10;
}
```

The no's in the program are :10
The keywords and identifiers are:
main is an identifier
int is a keyword
a is an identifier
b is an identifier
a is an identifier
b is an identifier
Special characters are : (){,;=+;}
Total no. of lines are:5nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$

10. write recursive descent parser for the grammar E->E+T E->T T->T*F T->F
F->(E)/id.

A recursive descent parser for this grammar requires eliminating left-recursion to prevent infinite loops.
he transformed grammar without left-recursion is:\
E --> T E'
E' --> + T E' / e
T --> F T'
T' --> * F T' / e
F --> (E) /id

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit recursiveparser.c

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

// Global variables
char input[100];
int i = 0;
char lookahead;

// Function prototypes
void E();
void Eprime();
void T();
void Tprime();
void F();
void match(char t);

void E() {
   T();
   Eprime();
}

void Eprime() {
   if (lookahead == '+') {
      match('+');
      T();
      Eprime();
   }
}

void T() {
   F();
   Tprime();
}

void Tprime() {
   if (lookahead == '*') {
      match('*');
```

```
        F();
        Tprime();
    }
}

void F() {
    if (lookahead == '(') {
        match('(');
        E();
        match(')');
    } else if (isalnum(lookahead)) {
        // Here we handle 'id' by matching any alphanumeric character
        match(lookahead);
    } else {
        printf("Syntax Error\n");
    }
}

void match(char t) {
    if (lookahead == t) {
        i++;
        lookahead = input[i];
    } else {
        printf("Syntax Error\n");
    }
}

int main() {
    printf("Enter an arithmetic expression: ");
    scanf("%s", input);

    lookahead = input[i];
    E();

    // Check if the whole string was successfully parsed
    if (lookahead == '\0') {
        printf("\nString is successfully parsed!\n");
    } else {
        printf("\nSyntax Error: String not fully parsed\n");
    }

    return 0;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc recursiveparser.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter an arithmetic expression: a+b*c

String is successfully parsed!
```

11. write recursive descent parser for the grammar S->(L) S->a L->L,S L->S

By removing left recursion from L, we introduce a new non-terminal
S --> (L) / a
L -->SL'
L'--> ,SL' /e

Creation: nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc pgm11.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Global variables for tracking input
char input[100];
int cursor = 0;

// Function prototypes corresponding to non-terminals
int S();
int L();
int L_dash();

// Helper function to match terminal characters
int match(char expected) {
    if (input[cursor] == expected) {
        cursor++;
        return 1;
    }
    return 0;
}

// Production rules for S -> (L) | a
int S() {
    if (input[cursor] == '(') {
        if (match('(')) {
            if (L()) {
                if (match(')')) {
                    return 1; // Successfully matched (L)
                }
            }
        }
        return 0;
    } else if (input[cursor] == 'a') {
        return match('a'); // Successfully matched 'a'
    }
    return 0;
}

// Production rules for L -> S L'
int L() {
    if (S()) {
        if (L_dash()) {
```

```
            return 1;
        }
    }
    return 0;
}
// Production rules for L' -> ,S L' | epsilon
int L_dash() {
    if (input[cursor] == ',') {
        if (match(',')) {
            if (S()) {
                if (L_dash()) {
                    return 1;
                }
            }
        }
        return 0;
    }
    // Epsilon conversion (null match always succeeds)
    return 1;
}

int main() {
    printf("Enter the string to parse: ");
    scanf("%s", input);

    cursor = 0; // Reset lookahead pointer

    // Begin parsing from the start symbol S
    if (S() && input[cursor] == '\0') {
        printf("\nString accepted successfully! Structural syntax is valid.\n");
    } else {
        printf("\nString rejected! Syntax error at index %d (character: '%c').\n", cursor, input[cursor]);
    }

    return 0;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc pgm11.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the string to parse: ((a)

String rejected! Syntax error at index 4 (character: ').
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the string to parse: (a),(a)

String rejected! Syntax error at index 3 (character: ',').
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the string to parse: (a,a)

String accepted successfully! Structural syntax is valid.
```

12. Write a C program to calculate first function for the grammar
E->E+T E->T T->T*F T->F F->(E)/id

Creation:nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit pgm12.c

```
#include<stdio.h>
#include<ctype.h>
void FIRST(char );
int count,n=0;
char prodn[10][10], first[10];
void main()
{
int i,choice;
char c,ch;
printf("How many productions ? :");
scanf("%d",&count);
printf("Enter %d productions epsilon= $ :\n\n",count);
for(i=0;i<count;i++)
scanf("%s%c",prodn[i],&ch);
do
{
n=0;
printf("Element :");
scanf("%c",&c);
FIRST(c);
printf("\n FIRST(%c)= { ",c);
for(i=0;i<n;i++)
printf("%c ",first[i]);
printf("}\n");

printf("press 1 to continue : ");
scanf("%d%c",&choice,&ch);
}
while(choice==1);
}
void FIRST(char c)
{
int j;
if(!(isupper(c)))first[n++]=c;
for(j=0;j<count;j++)
{
if(prodn[j][0]==c)
{
if(prodn[j][2]=='$') first[n++]='$';
else if(islower(prodn[j][2]))first[n++]=prodn[j][2];
else FIRST(prodn[j][2]);
}
}
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc pgm12.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
How many productions ? :8
Enter 8 productions epsilon= $ :

E=TD
D=+TD
D=$
T=FS
F=*FS
S=$
F=(E)
F=a
Element :E

 FIRST(E)= { * ( a }
press 1 to continue : 1
Element :D

 FIRST(D)= { + $ }
press 1 to continue : 1
Element :F

 FIRST(F)= { * ( a }
press 1 to continue : 1
Element :A

 FIRST(A)= { }
press 1 to continue : 1
Element :a

 FIRST(a)= { a }
press 1 to continue : 0
```

13. Write a YACC program to implement a top down parser for the given grammar.

```
 Creation:nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit top.l
%{
#include "y.tab.h"
#include <stdio.h>
%}

%%
[0-9]+          { return ID; }
[a-zA-Z]        { return ID; }
"+"             { return '+'; }
"-"             { return '-'; }
"*"             { return '*'; }
"/"             { return '/'; }
"("             { return '('; }
")"             { return ')'; }
[ \t]           ; /* skip whitespace */
\n              { return 0; } /* Logical end of input */
.               { return yytext[0]; }
%%
int yywrap()
 {
   return 1;
}

Creation:nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit top.y

%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
%}

%token ID

%%
/* Top-level start rule */
input: E { printf("SUCCESS: The string is valid according to the top-down grammar rules.\n");
          YYACCEPT; }
    ;

/* E -> T E' */
E: T Edist ;

/* E' -> + T E' | - T E' | epsilon */
Edist: '+' T Edist
    | '-' T Edist
    | /* epsilon */
    ;
```

```
/* T -> F T' */
T: F Tdist ;

/* T' -> * F T' | / F T' | epsilon */
Tdist: '*' F Tdist
    | '/' F Tdist
    | /* epsilon */
    ;

/* F -> ( E ) | id */
F: '(' E ')'
 | ID
 ;

%%

void yyerror(const char *s) {
   fprintf(stderr, "ERROR: Invalid Syntax (%s)\n", s);
}

int main() {
   printf("Enter the algebraic expression to parse: ");
   yyparse();
    return 0;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex top.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ yacc -dy top.y
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c y.tab.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the algebraic expression to parse: a+b*c
SUCCESS: The string is valid according to the top-down grammar rules.
```

13. Write a YACC program to evaluate algebraic expression.

Creation : nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ gedit cal.y

```
%{
#include<stdio.h>
#include<stdlib.h>
int yylex(void);
int yyerror(char *s);
%}
%token NAME NUMBER
%left GE LE NE EQ '<">"%'
%left '-"+'
%left '*"/'
%nonassoc UMINUS
%%
Statement :NAME '=' exp
        |exp {printf("=%d\n",$1);}
;
exp     : NUMBER {$$ == $1;}
        | exp '+' exp {$$ =$1 +$3;}
        | exp '-' exp {$$ =$1 - $3;}
        | exp '*' exp {$$ =$1 * $3;}
        | exp '/' exp {$$ =$1 / $3;}
        | exp '-' exp %prec UMINUS {$$ =-$2;}
        | '('exp')' {$$ =$2;}
        ;
%%

int main()
{
printf("Enter the expression:");
yyparse();
}
int yyerror(char *s)
{
printf("\n Error Occurred\n");
return 1;
}
```

Output:

```
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ lex cal.l
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ yacc -dy cal.y
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ cc lex.yy.c y.tab.c
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the expression:2+3
=5
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$ ./a.out
Enter the expression:2+3*7
=23
nnrg@nnrg-OptiPlex-Tower-7020:~/pavan$
```

ALTERNATIVE PROGRAM:

LEX PROGRAM

```
%{
#include <stdio.h>
#include "y.tab.h"
extern int yylval;
%}

%%
[0-9]+          { yylval = atoi(yytext); return NUMBER; }
[ \t]           { /* Ignore spaces and tabs */ }
\n              { return 0; /* Logical End of Input */ }
.               { return yytext[0]; /* Return operators or brackets exactly as
they are */ }
%%

int yywrap() {
    return 1;
}
```

YACC PROGRAM

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(char *s);
%}

%token NUMBER

/* Set operator precedence and left associativity */
%left '+' '-'
%left '*' '/'

%%
/* Grammar Rules */
statement: expr { printf("Result = %d\n", $1); return 0; }
         ;

expr: expr '+' expr    { $$ = $1 + $3; }
    | expr '-' expr    { $$ = $1 - $3; }
    | expr '*' expr    { $$ = $1 * $3; }
    | expr '/' expr    {
                         if($3 == 0) {
                             printf("Error: Division by zero!\n");
```

```
                            exit(1);
                        }
                        $$ = $1 / $3;
                    }
    | '(' expr ')'      { $$ = $2; }
    | NUMBER            { $$ = $1; }
    ;

%%
int main() {
    printf("Enter an algebraic expression: ");
    yyparse();
    return 0;
}

void yyerror(char *s) {
    fprintf(stderr, "Invalid Expression!\n");
}
```

OUTPUT: